

ALFRED — Addendum Cybersécurité

Nuances · Modèle STRIDE · Tests de charge · Gestion des incidents

Cognitive Products Lab · V1/V2 Prototype · Mai 2026 · Document complémentaire

Ce document est un addendum aux dossiers cybersécurité ALFRED déjà produits. Il apporte trois compléments essentiels : (1) les nuances nécessaires à la lecture honnête des scores obtenus, (2) l'application formelle du modèle de menaces STRIDE qui était cité comme référentiel mais non déployé visuellement, et (3) les éléments manquants identifiés — tests de charge sécurité et politique de gestion des incidents.

1. Nuances sur les scores de conformité

1.1 Lecture correcte des résultats

FORMULATION OFFICIELLE — À retenir pour tout entretien ou présentation

"Score 100/100 obtenu sur notre suite de tests interne couvrant 137 cas — il ne s'agit pas d'une certification externe ISO 27001, RGPD ou OWASP, mais d'une validation de couverture fonctionnelle sur un prototype en cours de développement."

Les scores présentés dans les dossiers cybersécurité ALFRED (100/100, OWASP 100%, RGPD 100%, ISO 27001 100%) sont le résultat de **tests automatisés internes** conçus et exécutés par l'équipe projet. Ils mesurent la couverture fonctionnelle des mécanismes implémentés — pas leur robustesse face à un audit tiers indépendant.

Ce que mesurent ces scores	Ce qu'ils ne mesurent pas
Couverture des 137 cas de test définis par l'équipe	Robustesse face à un attaquant réel non contraint
Présence et activation des mécanismes de sécurité	Efficacité sous charge ou en conditions adversariales réelles
Conformité fonctionnelle aux référentiels (OWASP, ISO, RGPD)	Certification formelle par un organisme accrédité
Cohérence de l'architecture Security by Design	Audit de code source par un tiers indépendant
Validation du pipeline Zero Trust sur scénarios prédéfinis	Red team externe, penetration testing professionnel

1.2 Positionnement honnête du prototype

CE QUE LE PROJET DEMONTRE REELLEMENT

Une architecture Security by Design cohérente et documentée.

Une maîtrise des concepts Zero Trust, RBAC, MFA, chiffrement, audit trail.

Une approche gouvernance mesurable avec dashboard et KPI.
Une couverture fonctionnelle validée sur 137 cas de test internes.
Une démarche de conformité structurée — référentiel OWASP, ISO 27001, RGPD — appliquée à un prototype, non certifiée.

LIMITES A ASSUMER CLAIREMENT
Pas de certification tierce indépendante — les scores sont autoévalués.
Prototype actif : plusieurs fonctionnalités critiques restent à industrialiser (rotation des clés, révocation appareils, corrélation événements, behavioral ML).
Pas de test de charge sécurité réalisé à ce stade (voir Section 3).
Pas de red team externe ni de pentest professionnel conduit sur le système.

2. Modèle de menaces STRIDE — Application formelle

STRIDE est cité comme référentiel dans les dossiers ALFRED mais n'avait pas été appliqué formellement avec une cartographie menace par menace. Cette section comble cet écart.

2.1 Rappel de la méthode STRIDE

STRIDE est un modèle de modélisation des menaces développé par Microsoft. Il classe les menaces en 6 catégories et permet de s'assurer qu'aucune famille d'attaque n'est oubliée lors de la conception d'un système.

Lettre	Menace
S — Spoofing	Usurpation d'identité
T — Tampering	Altération des données
R — Repudiation	Déni d'une action réalisée
I — Information Disclosure	Divulgateion non autorisée d'informations
D — Denial of Service	Perturbation de la disponibilité
E — Elevation of Privilege	Escalade de privilèges

2.2 Application STRIDE au système ALFRED

Le tableau ci-dessous applique chaque catégorie STRIDE au contexte spécifique d'ALFRED, en précisant l'exemple d'attaque concret, le contrôle mis en place et son statut d'implémentation actuel.

Menace STRIDE	Exemple concret ALFRED	Contrôle associé	Statut
S — Spoofing (usurpation identité)	Un attaquant se fait passer pour l'utilisateur légitime	Authentification bcrypt + MFA + session token	Implémenté

T — Tampering (altération données)	Modification des fichiers mémoire JSON ou logs	Audit trail JSONL + chiffrement Fernet	Implémenté
R — Repudiation (non-répudiation)	Un utilisateur nie avoir émis une commande vocale	Audit trail horodaté UTC + logs sécurité	Implémenté
I — Info Disclosure (fuite information)	Exfiltration de la mémoire conversationnelle	Chiffrement local + output_filter + local-first	Implémenté
D — Denial of Service (dénier de service)	Saturation du pipeline par requêtes massives	Rate limiting — seuil 100 req/min	Partiel — sans test de charge
E — Elevation of Privilege (escalade privilèges)	Un utilisateur USER tente d'accéder à des fonctions ADMIN	RBAC 7 rôles + policy_engine + Zero Trust	Implémenté

2.3 Flux de données analysés

Le modèle STRIDE s'applique aux flux suivants dans ALFRED :

Flux	Menaces STRIDE applicables	Protection principale
Utilisateur → input_validator	S, T, I	Validation 25 patterns + Unicode NFKC
input_validator → threat_detector	T, D	Score menace — seuil configurable
authenticator → session	S, R	bcrypt + TOTP + audit trail
RBAC → policy_engine	E	7 rôles + least privilege + Zero Trust
Mémoire / données persistantes	T, I	Fernet + chiffrement rétroactif + local-first
output_filter → réponse	I	Masquage clés API, JWT, tokens

3. Tests de charge sécurité — État et roadmap

3.1 Situation actuelle

ECART IDENTIFIÉ

Les tests de sécurité actuels (137/137) valident le comportement fonctionnel des mécanismes de protection (blocage injections, MFA, RBAC, chiffrement) sur des requêtes unitaires.

Aucun test de charge sécurité n'a été conduit à ce stade : le comportement du pipeline Zero Trust sous volume élevé de requêtes simultanées n'est pas encore mesuré.

3.2 Définition — Tests de charge sécurité

Un test de charge sécurité (security load testing) vise à vérifier que les mécanismes de protection maintiennent leur efficacité sous contrainte de volume et de concurrence, et à identifier les points de rupture avant qu'un attaquant ne les exploite.

Type de test	Objectif pour ALFRED
Test de charge nominale	Valider que le pipeline Zero Trust répond en < 200ms pour 50 req/s
Test de saturation (rate limiting)	Vérifier que le rate limiter bloque à partir de 100 req/min par utilisateur
Test de flood injection	Envoyer 10 000 requêtes malveillantes simultanées — mesurer taux de blocage maintenu
Test brute-force MFA	Simuler 1 000 tentatives d'authentification concurrentes — vérifier verrouillage
Test épuisement mémoire	Vérifier l'absence de memory leak sur sessions longues et volumineuses
Test dégradation gracieuse	Valider que le système refuse proprement en cas de surcharge (fail-safe)

3.3 Roadmap tests de charge — Priorités

Priorité	Test	Outil recommandé	Phase cible
Critique	Flood injection + rate limiting	locust / pytest-benchmark	V2
Critique	Brute-force MFA concurrent	locust + script custom	V2
Élevée	Pipeline Zero Trust sous charge	locust + profiler Python	V2
Élevée	Dégradation gracieuse (fail-safe)	script pytest custom	V2
Moyenne	Memory leak longue durée	memory_profiler	V3

4. Politique de gestion des incidents — Procédure formelle

4.1 Situation actuelle

ECART IDENTIFIÉ
Le projet dispose d'un fichier incident_register.json et d'un module incident_manager.py qui enregistrent les incidents techniques.
Il n'existait pas, jusqu'à ce document, de politique de gestion des incidents formalisée : pas de niveaux de sévérité définis, pas de procédure de réponse documentée, pas de délais de traitement, pas de processus de clôture.

4.2 Classification des incidents

Niveau	Sévérité	Critères	Exemple ALFRED	Délai réponse
--------	----------	----------	----------------	---------------

P1	Critique	Atteinte données utilisateur, compromission système	Exfiltration mémoire, bypass Zero Trust, compromission MFA	Immédiat < 15 min
P2	Élevée	Tentative d'attaque réussie partiellement, anomalie sécurité	Escalade de privilèges détectée, injection partiellement passée	< 1 heure
P3	Moyenne	Tentative bloquée répétée, comportement suspect	Flood injection bloqué, brute-force détecté	< 4 heures
P4	Faible	Anomalie non critique, alerte monitoring	Latence anormale pipeline, log rate unusual	< 24 heures

4.3 Procédure de réponse aux incidents — 5 phases

1. DÉTECTION

- Source : audit_trail + security_logger + dashboard cybersécurité
- Déclencheurs : alerte automatique score sécurité, log anomalie, tentative bloquée répétée
- Action : enregistrement incident dans incident_register.json avec horodatage UTC

2. QUALIFICATION

- Analyser le log pour déterminer le niveau de sévérité (P1 à P4)
- Identifier le composant affecté (input_validator, MFA, RBAC, chiffrement...)
- Décider si le système doit passer en mode dégradé ou être isolé

3. CONTAINMENT

- P1/P2 : isolement immédiat du composant affecté si possible
- Activation du mode fail-safe : refus par défaut de toutes les requêtes non certifiées
- Révocation de la session ou de l'appareil suspect via device_registry
- Préservation des logs pour analyse forensique

4. REMEDIATION

- Identification de la cause racine (root cause analysis)
- Correction du vecteur d'attaque ou de la faille exploitée
- Mise à jour des patterns input_validator si nouvelle injection détectée
- Rotation des clés si compromission chiffrement suspectée
- Tests de régression sur le composant corrigé

5. CLOTURE & RETEX

- Mise à jour de l'incident dans incident_register.json : statut 'fermé', cause, actions
- Mise à jour de la matrice des risques si nouveau risque identifié
- Mise à jour du dashboard sécurité
- Documentation du retour d'expérience (RETEX) pour amélioration continue
- Vérification que le score sécurité global est restauré

4.4 Format incident_register.json — Structure cible

La structure du fichier incident_register.json doit être enrichie pour supporter la politique ci-dessus :

```
{ "incident_id": "INC-2026-001", "timestamp_utc": "2026-05-15T14:32:10Z", "severity": "P2",  
  "component": "input_validator", "attack_type": "prompt_injection", "input_blocked": "Ignore  
previous instructions...", "status": "closed", "response_time_minutes": 22, "root_cause":  
  "pattern non couvert – ajout pattern #26", "actions_taken": ["pattern mis à jour", "tests  
regression OK"], "retex": "Renforcer détection jailbreak multi-étapes" }
```

5. Synthèse — Maturité réelle du projet

Domaine	Niveau réel	Commentaire honnête
Architecture Zero Trust	Avancé	Pipeline complet, PDP/PEP, 7 rôles RBAC — opérationnel
Sécurité applicative (OWASP)	Avancé	25 patterns, 10/10 OWASP couverts fonctionnellement
MFA & Authentification	Opérationnel	bcrypt + TOTP + device + comportemental
Chiffrement	Opérationnel	Fernet, rétroactif, secrets hors Git
Modèle de menaces STRIDE	Formalisé ici	Appliqué dans ce document — cité mais absent des dossiers précédents
Tests de charge sécurité	Non réalisés	Roadmap V2 — locust + pytest-benchmark
Gestion des incidents	Procédure formalisée ici	Fichier existait — politique P1→P4 + 5 phases ajoutée ici
Rotation des clés	Non implémenté	Priorité critique V2
Corrélation SIEM / ML	Non implémenté	Priorité V2/V3
Certification externe	Non applicable	Prototype — autoévaluation interne uniquement

CONCLUSION — CE QUE CE DOSSIER APPORTE

Ce document complète les dossiers cybersécurité ALFRED en apportant la rigueur et l'honnêteté nécessaires à une présentation professionnelle.

Les scores 100/100 sont désormais correctement contextualisés : validation fonctionnelle interne sur prototype, pas une certification.

STRIDE est formellement appliqué : 6 familles de menaces cartographiées sur les flux ALFRED avec statut d'implémentation par menace.

Les tests de charge sécurité sont définis et planifiés en roadmap V2.

La politique de gestion des incidents est formalisée : 4 niveaux P1→P4, 5 phases de réponse, format incident_register enrichi.

Cognitive Products Lab · Mai 2026 · Document confidentiel · Addendum au dossier cybersécurité ALFRED